
Projet Big Data

Analyse de Données de Vente

Apache Spark (PySpark) & Hadoop HDFS sous Docker

Dataset : Online Retail (UCI) — $\approx$ 541 909 transactions

Membres du groupe

Alaa Belhadj

Zackaria Ajli

Oussema Jebaari

Aziz Ben Guirat

Mohamed Hamdaoui

Ibtihal El Bannouri

Table des matières

1	Introduction et contexte	2
1.1	Jeu de données	2
2	Architecture de la solution	2
3	Mise en route : de docker compose up à l'exécution	2
3.1	Préparation des données	2
3.2	Démarrage de la stack	3
3.3	Accès à Jupyter	3
4	Partie 1 — Application d'analyse avec Spark	4
4.1	Initialisation de la session Spark	4
4.2	Chargement des données	4
4.3	Nettoyage et transformation	5
4.4	Analyses des ventes et des produits	6
4.5	Indicateurs clés (KPIs)	10
5	Partie 2 — Stockage sur Hadoop HDFS	10
6	Conclusion	12

1 Introduction et contexte

Une entreprise de commerce en ligne souhaite exploiter l'historique de ses ventes afin de mieux comprendre son activité, d'analyser le comportement de ses clients et d'identifier des opportunités d'amélioration de ses performances commerciales.

L'objectif de ce projet est de développer une **solution d'analyse de données** reposant sur **Apache Spark** pour traiter efficacement plusieurs centaines de milliers de transactions, puis de stocker le jeu de données sur un cluster **Hadoop HDFS**. L'ensemble est conteneurisé avec **Docker Compose**.

1.1 Jeu de données

Le projet s'appuie sur le *Online Retail Dataset* de l'UCI Machine Learning Repository¹, qui contient environ **541 909** transactions d'un site de vente en ligne britannique entre décembre 2010 et décembre 2011. Chaque ligne décrit un article acheté : numéro de facture, code et description du produit, quantité, date, prix unitaire, identifiant client et pays.

Colonne	Type	Description
InvoiceNo	string	Numéro de facture
StockCode	string	Code produit
Description	string	Libellé du produit
Quantity	int	Quantité achetée
InvoiceDate	date	Date et heure de la transaction
UnitPrice	double	Prix unitaire (£)
CustomerID	int	Identifiant client
Country	string	Pays du client

TABLE 1 – Schéma du jeu de données Online Retail.

2 Architecture de la solution

La solution est composée de trois services orchestrés par `docker-compose.yml` sur un réseau commun `bigdata-net` :

Service	Image Docker	Rôle	Interface
namenode	bde2020/hadoop-namenode	Maître HDFS (métadonnées)	localhost :9870
datanode	bde2020/hadoop-datanode	Stockage des blocs HDFS	localhost :9864
spark	jupyter/pyspark-notebook	Spark + Jupyter (PySpark)	localhost :8888

TABLE 2 – Les trois conteneurs de la stack Big Data.

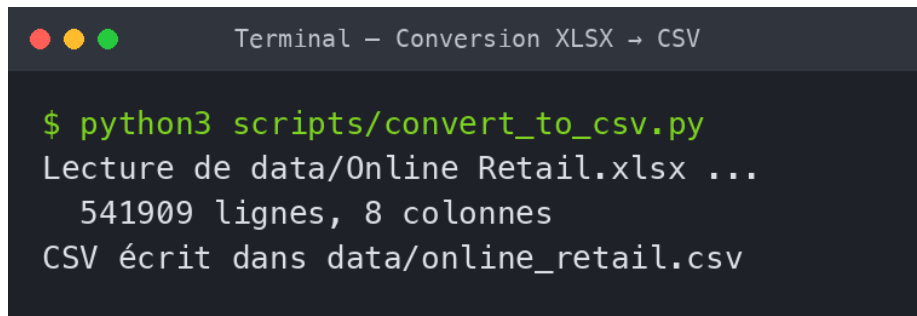
Le dossier local `data/` (contenant le CSV) est monté dans le conteneur Spark, et le conteneur Spark écrit ses résultats sur le cluster HDFS via l'adresse `hdfs://namenode:9000`.

3 Mise en route : de docker compose up à l'exécution

3.1 Préparation des données

Le dataset est fourni au format Excel (`.xlsx`). Un script Python le convertit en CSV, format directement exploitable par Spark.

1. <https://archive.ics.uci.edu/dataset/352/online+retail>



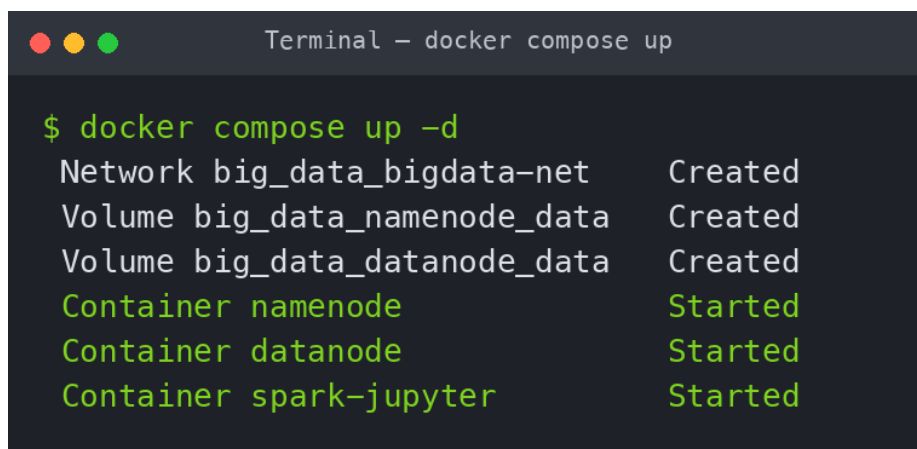
```
Terminal - Conversion XLSX → CSV

$ python3 scripts/convert_to_csv.py
Lecture de data/Online Retail.xlsx ...
  541909 lignes, 8 colonnes
CSV écrit dans data/online_retail.csv
```

FIGURE 1 – Conversion du fichier `.xlsx` en `.csv`.

3.2 Démarrage de la stack

La commande `docker compose up -d` construit le réseau, les volumes persistants et démarre les trois conteneurs.

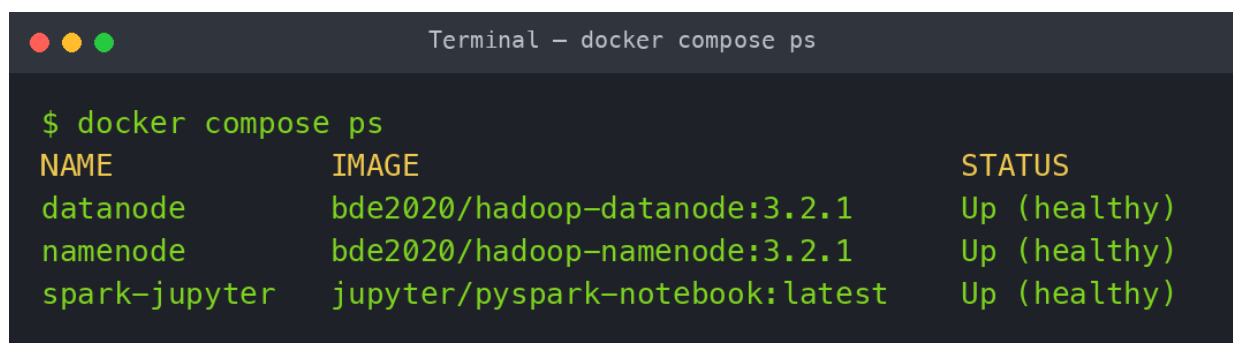


```
Terminal - docker compose up

$ docker compose up -d
Network big_data_bigdata-net      Created
Volume big_data_namenode_data     Created
Volume big_data_datanode_data     Created
Container namenode                Started
Container datanode                Started
Container spark-jupyter           Started
```

FIGURE 2 – Démarrage des conteneurs avec `docker compose up`.

On vérifie ensuite que les trois services sont sains (*healthy*).



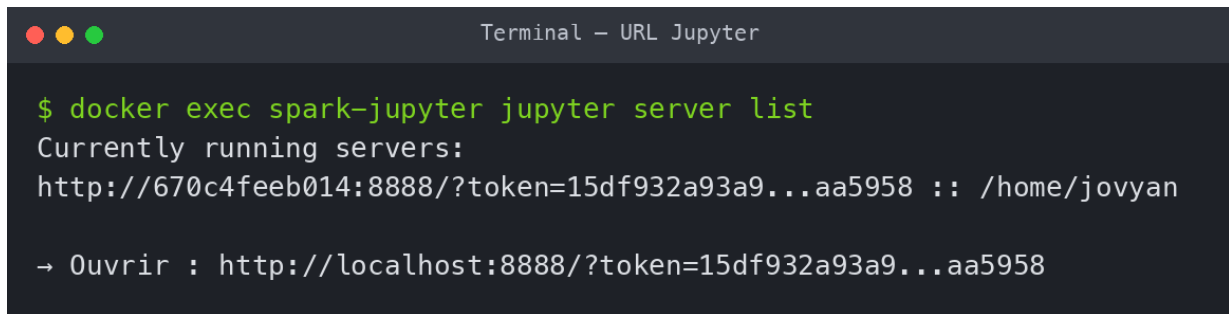
```
Terminal - docker compose ps

$ docker compose ps
NAME                IMAGE                                STATUS
datanode            bde2020/hadoop-datanode:3.2.1      Up (healthy)
namenode            bde2020/hadoop-namenode:3.2.1      Up (healthy)
spark-jupyter       jupyter/pyspark-notebook:latest    Up (healthy)
```

FIGURE 3 – État des conteneurs : `docker compose ps`.

3.3 Accès à Jupyter

On récupère l'URL d'accès à Jupyter (avec son token de sécurité).



```
Terminal – URL Jupyter

$ docker exec spark-jupyter jupyter server list
Currently running servers:
http://670c4feeb014:8888/?token=15df932a93a9...aa5958 :: /home/jovyan

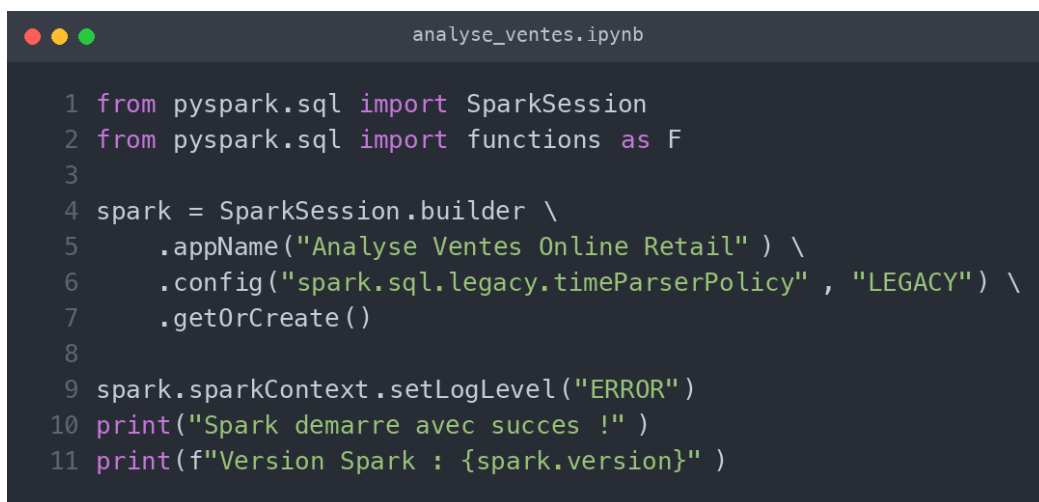
→ Ouvrir : http://localhost:8888/?token=15df932a93a9...aa5958
```

FIGURE 4 – Récupération de l'URL Jupyter.

4 Partie 1 — Application d'analyse avec Spark

4.1 Initialisation de la session Spark

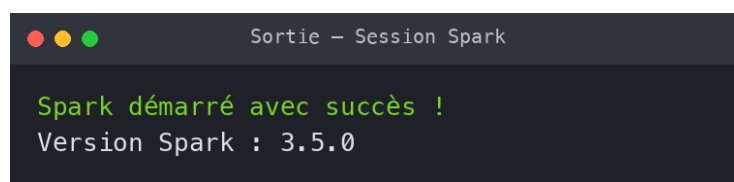
On crée une `SparkSession`, point d'entrée de toute application PySpark.



```
analyse_ventes.ipynb

1 from pyspark.sql import SparkSession
2 from pyspark.sql import functions as F
3
4 spark = SparkSession.builder \
5     .appName("Analyse Ventes Online Retail" ) \
6     .config("spark.sql.legacy.timeParserPolicy" , "LEGACY") \
7     .getOrCreate()
8
9 spark.sparkContext.setLogLevel("ERROR")
10 print("Spark démarre avec succès !")
11 print(f"Version Spark : {spark.version}")
```

FIGURE 5 – Code — création de la session Spark.



```
Sortie – Session Spark

Spark démarré avec succès !
Version Spark : 3.5.0
```

FIGURE 6 – Sortie — Spark 3.5.0 démarré.

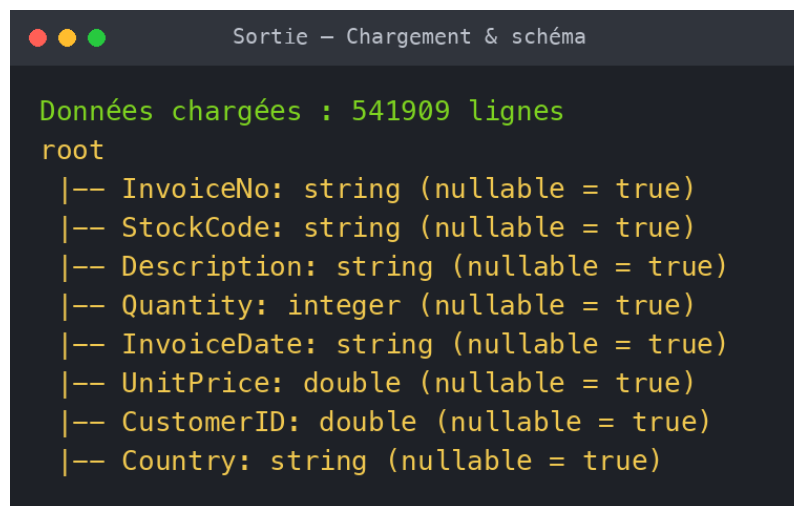
4.2 Chargement des données

Le fichier CSV est chargé dans un `DataFrame` Spark avec inférence du schéma.



```
1 df = spark.read \  
2     .option("header", "true") \  
3     .option("inferSchema", "true") \  
4     .option("multiLine", "true") \  
5     .option("escape", "\\") \  
6     .csv("/home/jovyan/data/online_retail.csv" )  
7  
8 print(f"Donnees chargees : {df.count()} lignes" )  
9 df.printSchema()
```

FIGURE 7 – Code — chargement du CSV.



```
Données chargées : 541909 lignes  
root  
|-- InvoiceNo: string (nullable = true)  
|-- StockCode: string (nullable = true)  
|-- Description: string (nullable = true)  
|-- Quantity: integer (nullable = true)  
|-- InvoiceDate: string (nullable = true)  
|-- UnitPrice: double (nullable = true)  
|-- CustomerID: double (nullable = true)  
|-- Country: string (nullable = true)
```

FIGURE 8 – Sortie — 541 909 lignes chargées et schéma inféré.

4.3 Nettoyage et transformation

Le jeu de données brut contient des valeurs manquantes (notamment 135 080 lignes sans `CustomerID`) et des lignes correspondant à des retours (quantités négatives). On applique donc plusieurs traitements :

- suppression des lignes sans identifiant client ;
- suppression des quantités et prix négatifs ou nuls ;
- suppression des doublons ;
- ajout d'une colonne $\text{TotalPrice} = \text{Quantity} \times \text{UnitPrice}$;
- conversion de la date et extraction de l'année et du mois.

Sortie – Valeurs nulles

Valeurs nulles par colonne :

InvoiceNo	StockCode	Description	Quantity	UnitPrice	CustomerID	Country
0	0	1454	0	0	135080	0

FIGURE 9 – Sortie — comptage des valeurs nulles par colonne.

analyse_ventes.ipynb

```

1 df_clean = df \
2     .filter(F.col("CustomerID").isNotNull()) \
3     .filter(F.col("Quantity") > 0) \
4     .filter(F.col("UnitPrice") > 0) \
5     .dropDuplicates()
6
7 df_clean = df_clean.withColumn("CustomerID", F.col("CustomerID").cast("int"))
8 df_clean = df_clean.withColumn("TotalPrice",
9     F.round(F.col("Quantity") * F.col("UnitPrice"), 2))
10 df_clean = df_clean.withColumn("InvoiceDate",
11     F.to_timestamp("InvoiceDate", "yyyy-MM-dd HH:mm:ss"))
12 df_clean = df_clean.withColumn("Month", F.month("InvoiceDate"))
13 df_clean = df_clean.withColumn("Year", F.year("InvoiceDate"))

```

FIGURE 10 – Code — nettoyage et transformation.

Sortie – Données nettoyées

Après nettoyage : 392692 lignes (sur 541909)

InvoiceNo	StockCode	Description	Quantity	CustomerID	TotalPrice	Month	Year
536408	84879	ASSORTED COLOUR BIRD ORNAMENT	8	14307	13.52	12	2010
536409	22074	6 RIBBONS SHIMMERING PINKS	1	17908	1.65	12	2010
536409	90199C	5 STRAND GLASS NECKLACE CRYST.	2	17908	12.7	12	2010

FIGURE 11 – Sortie — 392 692 lignes après nettoyage.

4.4 Analyses des ventes et des produits

Sortie – CA total

Chiffre d'affaires total : 8887208.89 £

FIGURE 12 – Chiffre d'affaires total : 8 887 208.89 £.

Chiffre d'affaires total.

Sortie – Top produits (quantité)

Top 10 produits les plus vendus (quantité) :

StockCode	Description	Total_Vendu
23843	PAPER CRAFT , LITTLE BIRDIE	80995
23166	MEDIUM CERAMIC TOP STORAGE JAR	77916
84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	54319
85099B	JUMBO BAG RED RETROSPOT	46078
85123A	WHITE HANGING HEART T-LIGHT HOLDER	36706
84879	ASSORTED COLOUR BIRD ORNAMENT	35263
21212	PACK OF 72 RETROSPOT CAKE CASES	33670
22197	POPCORN HOLDER	30919
23084	RABBIT NIGHT LIGHT	27153
22492	MINI PAINT SET VINTAGE	26076

FIGURE 13 – Top 10 des produits par quantité vendue.

Sortie – Top produits (CA)

Top 10 produits par chiffre d'affaires :

StockCode	Description	CA
23843	PAPER CRAFT , LITTLE BIRDIE	168469.6
22423	REGENCY CAKESTAND 3 TIER	142264.75
85123A	WHITE HANGING HEART T-LIGHT HOLDER	100392.1
85099B	JUMBO BAG RED RETROSPOT	85040.54
23166	MEDIUM CERAMIC TOP STORAGE JAR	81416.73
POST	POSTAGE	77803.96
47566	PARTY BUNTING	68785.23
84879	ASSORTED COLOUR BIRD ORNAMENT	56413.03
M	Manual	53419.93
23084	RABBIT NIGHT LIGHT	51251.24

FIGURE 14 – Top 10 des produits par chiffre d'affaires.

Produits les plus vendus.

Répartition géographique. Le marché est très largement dominé par le Royaume-Uni (>80 % du CA).



FIGURE 15 – CA par pays (Top 10).

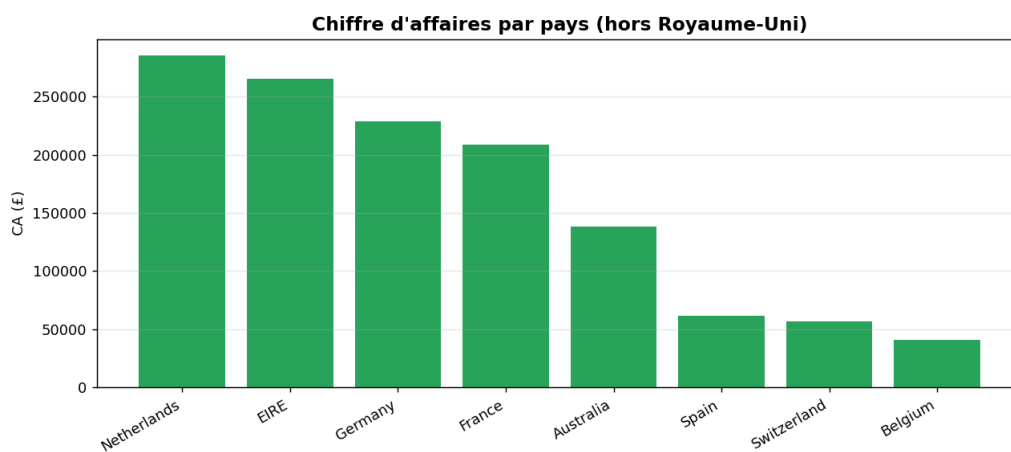


FIGURE 16 – Visualisation du CA par pays (hors Royaume-Uni).

Évolution temporelle. On observe une forte croissance du CA à l'approche de la période de Noël 2011.

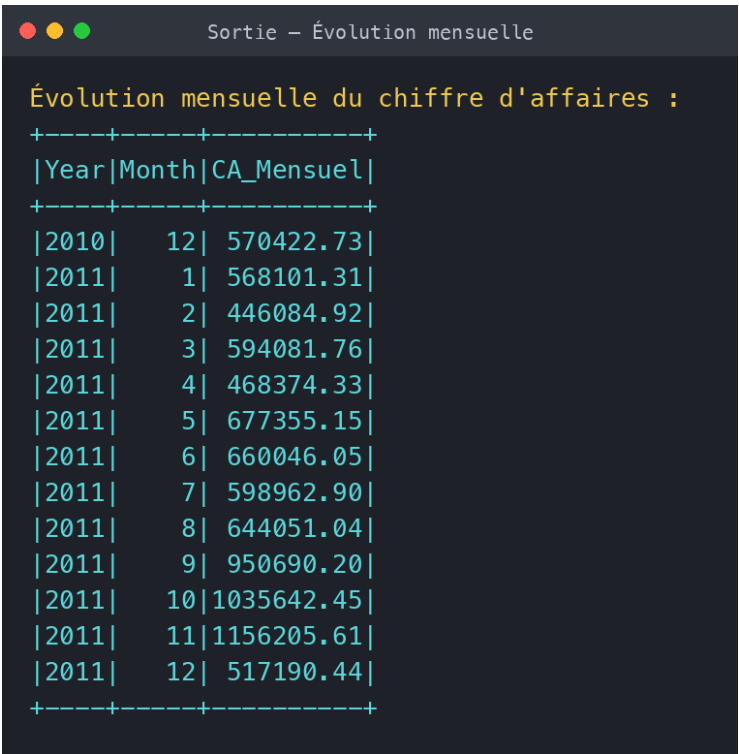


FIGURE 17 – Évolution mensuelle du CA.

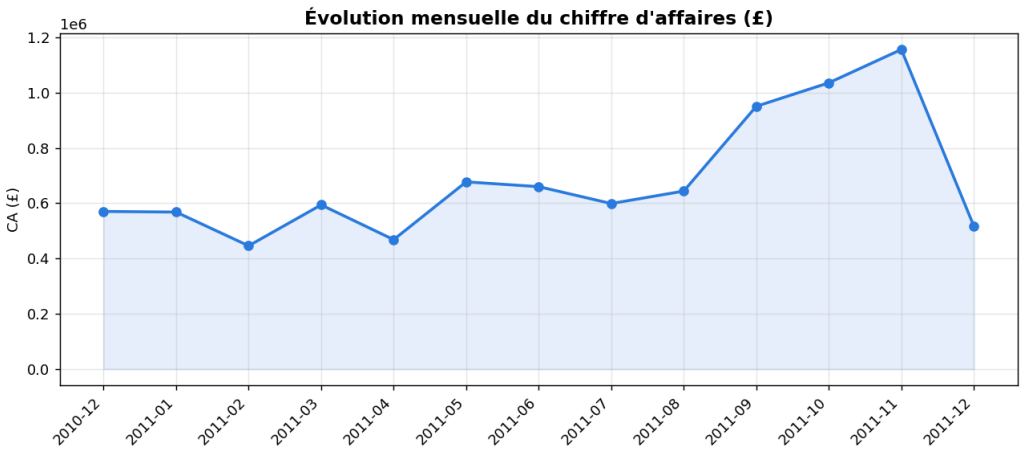
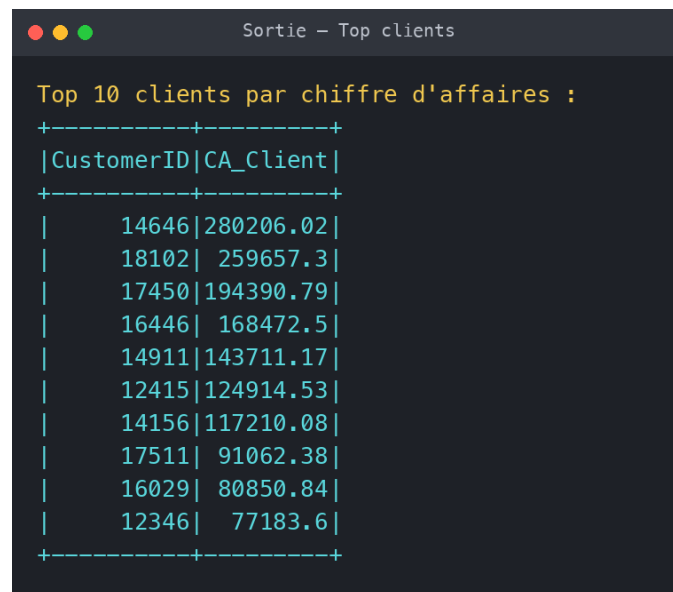


FIGURE 18 – Visualisation de l'évolution mensuelle du CA.



```

Sortie - Top clients

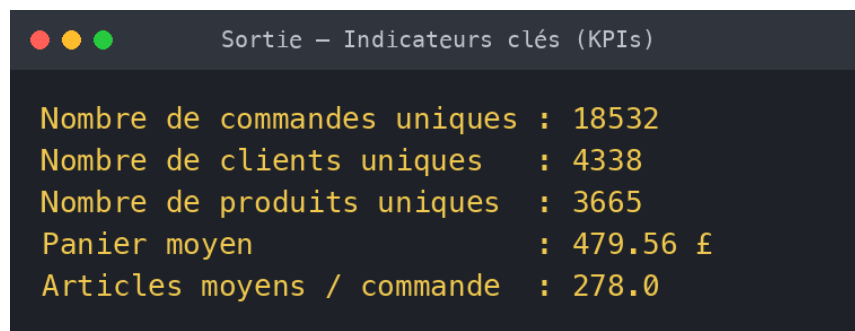
Top 10 clients par chiffre d'affaires :
+-----+-----+
|CustomerID|CA_Client|
+-----+-----+
|      14646|280206.02|
|      18102| 259657.3|
|      17450|194390.79|
|      16446| 168472.5|
|      14911|143711.17|
|      12415|124914.53|
|      14156|117210.08|
|      17511| 91062.38|
|      16029| 80850.84|
|      12346| 77183.6|
+-----+-----+

```

FIGURE 19 – Top 10 des clients par chiffre d'affaires.

Meilleurs clients.

4.5 Indicateurs clés (KPIs)



```

Sortie - Indicateurs clés (KPIs)

Nombre de commandes uniques : 18532
Nombre de clients uniques   : 4338
Nombre de produits uniques  : 3665
Panier moyen                 : 479.56 £
Articles moyens / commande  : 278.0

```

FIGURE 20 – Indicateurs synthétiques de l'activité.

Indicateur	Valeur
Chiffre d'affaires total	8 887 208.89 £
Commandes uniques	18 532
Clients uniques	4 338
Produits uniques	3 665
Panier moyen par commande	479.56 £
Articles moyens par commande	278.0

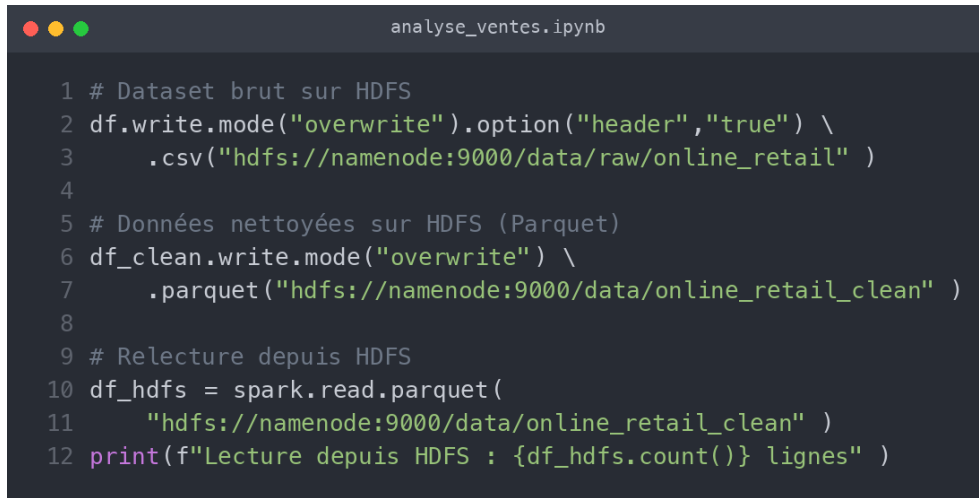
TABLE 3 – Synthèse des indicateurs calculés.

5 Partie 2 — Stockage sur Hadoop HDFS

Conformément à l'énoncé, le jeu de données est stocké sur le cluster HDFS. Deux versions sont écrites depuis Spark :

- le **dataset brut** dans `/data/raw/online_retail` (CSV) ;

- les **données nettoyées** dans `/data/online_retail_clean` (format colonne Parquet, compressé Snappy, 8 partitions).

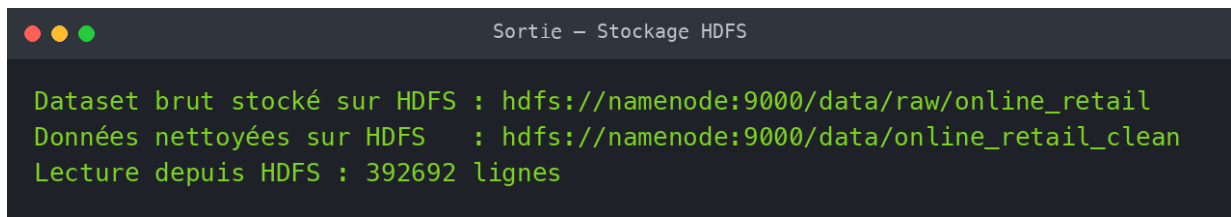


```

1 # Dataset brut sur HDFS
2 df.write.mode("overwrite").option("header","true") \
3   .csv("hdfs://namenode:9000/data/raw/online_retail" )
4
5 # Données nettoyées sur HDFS (Parquet)
6 df_clean.write.mode("overwrite") \
7   .parquet("hdfs://namenode:9000/data/online_retail_clean" )
8
9 # Relecture depuis HDFS
10 df_hdfs = spark.read.parquet(
11   "hdfs://namenode:9000/data/online_retail_clean" )
12 print(f"Lecture depuis HDFS : {df_hdfs.count()} lignes" )

```

FIGURE 21 – Code — écriture et relecture sur HDFS.



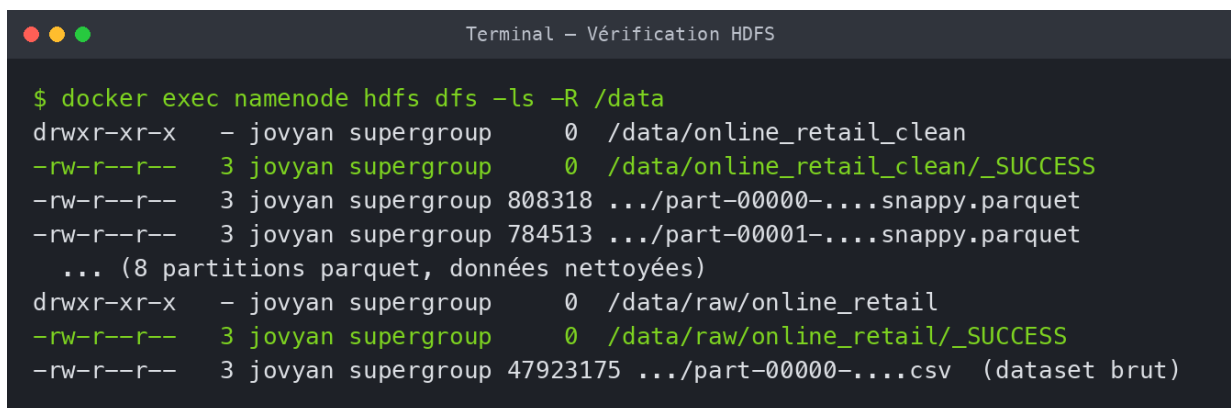
```

Dataset brut stocké sur HDFS : hdfs://namenode:9000/data/raw/online_retail
Données nettoyées sur HDFS   : hdfs://namenode:9000/data/online_retail_clean
Lecture depuis HDFS : 392692 lignes

```

FIGURE 22 – Sortie — confirmation des écritures HDFS.

On vérifie enfin l'arborescence stockée sur le cluster :



```

$ docker exec namenode hdfs dfs -ls -R /data
drwxr-xr-x  - jovyan supergroup    0 /data/online_retail_clean
-rw-r--r--  3 jovyan supergroup    0 /data/online_retail_clean/_SUCCESS
-rw-r--r--  3 jovyan supergroup 808318 .../part-00000-....snappy.parquet
-rw-r--r--  3 jovyan supergroup 784513 .../part-00001-....snappy.parquet
... (8 partitions parquet, données nettoyées)
drwxr-xr-x  - jovyan supergroup    0 /data/raw/online_retail
-rw-r--r--  3 jovyan supergroup    0 /data/raw/online_retail/_SUCCESS
-rw-r--r--  3 jovyan supergroup 47923175 .../part-00000-....csv (dataset brut)

```

FIGURE 23 – Contenu du cluster HDFS (`hdfs dfs -ls -R /data`).

La réplication est fixée à 1 (un seul DataNode), et l'interface web du NameNode (<http://localhost:9870>) permet de superviser le cluster et de naviguer dans le système de fichiers.

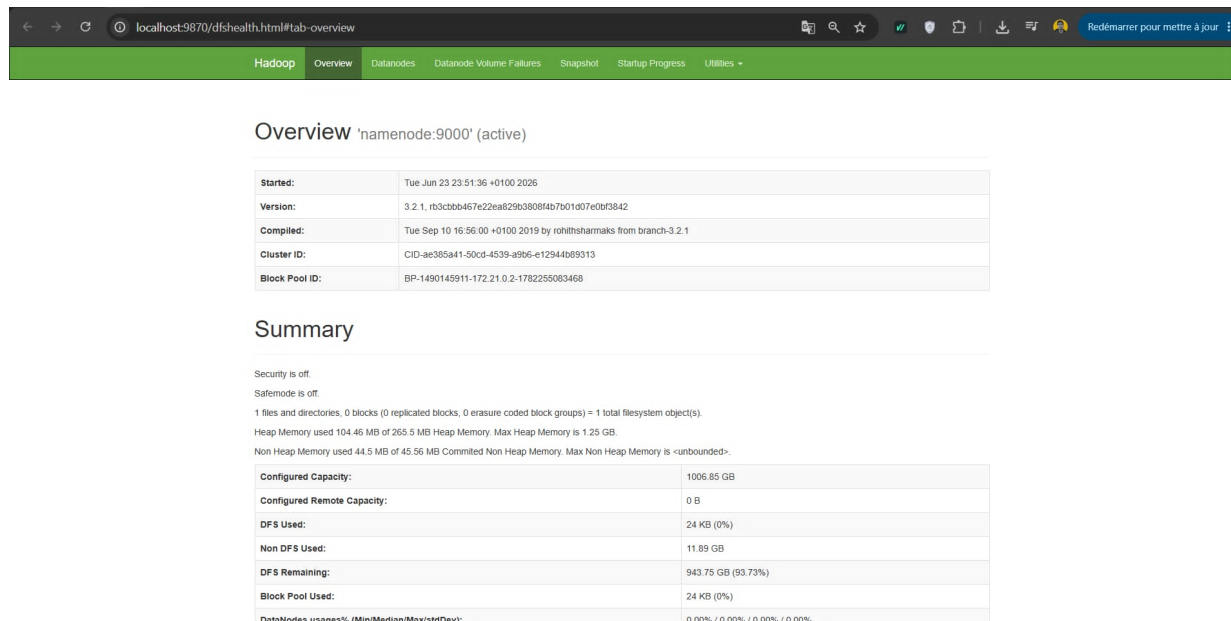


FIGURE 24 – Interface web du NameNode Hadoop (localhost:9870) : vue d'ensemble du cluster HDFS (version 3.2.1, *Safemode off*, capacité et utilisation du système de fichiers).

6 Conclusion

Ce projet a permis de mettre en place une chaîne complète de traitement Big Data : ingestion d'un dataset de plus de 540 000 transactions, nettoyage et transformation distribués avec **PySpark**, calcul d'analyses et d'indicateurs métier pertinents, puis stockage durable sur un cluster **Hadoop HDFS**, le tout reproductible grâce à **Docker Compose**.

Les analyses montrent une activité fortement concentrée sur le Royaume-Uni, une saisonnalité marquée (pic à l'approche de Noël) et une dépendance à un petit nombre de clients à forte valeur. Comme **extension**, la solution pourrait être étendue au traitement en temps réel (*streaming*) à l'aide de Spark Structured Streaming alimenté par Apache Kafka.